

04_conformal_calibration

July 18, 2026

1 04 — Phase 2: Conformal calibration + coverage validation

Purpose: run the real O*NET/OEWS pipeline through a proper train/calibration/test split, calibrate a split-conformal prediction interval on the trained baseline model, and validate coverage — marginally, and conditionally by Job Zone, SOC major group, and percentile level. If any subgroup shows real miscoverage, apply Mondrian (group-conditional) calibration as a direct response and report coverage before/after.

This is the real-data counterpart to `tests/test_conformal.py`'s synthetic checks — those already confirmed the math (quantile formula, coverage-on-average, interval validity, split disjointness, Mondrian group-specificity) is correct. This notebook is where we find out what the real data actually shows.

```
[1]: import sys
from pathlib import Path
# Notebook runs from notebooks/, so add project root to the path
project_root = Path.cwd().parent
sys.path.insert(0, str(project_root))

import numpy as np
import pandas as pd

from src.data.onet_loader import (
    load_occupation_data,
    load_essential_skills,
    load_transferable_skills,
    load_job_zones,
    build_full_skill_importance_matrix,
)
from src.data.oews_loader import load_national_wages
from src.features.occupation_features import (
    build_occupation_feature_table,
    build_training_rows,
    clean_training_rows,
)
from src.models.wage_model import (
    occupation_train_cal_test_split,
    occupation_train_test_split,
```

```

    train_baseline_model,
    evaluate,
)
from src.models.conformal import (
    calibrate,
    calibrate_by_group,
    predict_interval,
    predict_interval_by_group,
    evaluate_coverage,
    evaluate_conditional_coverage,
    soc_major_group,
)

```

1.1 1. Rebuild the Phase 1 pipeline

Identical to notebook 03 — load data, build features, clean training rows. No changes here; conformal calibration sits ON TOP of the existing pipeline, not inside it.

```

[2]: occ = load_occupation_data()
    skills = load_essential_skills()
    transferable = load_transferable_skills()
    job_zones = load_job_zones()
    wages = load_national_wages()

    skill_matrix = build_full_skill_importance_matrix(skills, transferable)
    feature_table = build_occupation_feature_table(occ, skill_matrix, job_zones,
    ↪wages)
    training_rows = build_training_rows(feature_table, skill_cols=list(skill_matrix.
    ↪columns))
    cleaned_training_rows = clean_training_rows(training_rows,
    ↪skill_cols=list(skill_matrix.columns))

    feature_cols = list(skill_matrix.columns) + ["job_zone", "percentile"]

    print(f"Final training set: {cleaned_training_rows.shape}")
    print(f"Unique occupations: {cleaned_training_rows['onetsoc_code'].nunique()}")

```

[oews_loader] 7 rows have suppressed wage data - kept as NaN, not dropped.
Dropped 470 rows (94 occupations) missing all skill values, out of 4780 total rows.

No job_zone imputation needed.

Final training set: (4310, 39)

Unique occupations: 862

1.2 2. Three-way occupation split

Using `occupation_train_cal_test_split()` with the SAME `test_size=0.2`, `random_state=42` as notebook 03's two-way split, so `test_df` here should be IDENTICAL

to notebook 03's test set — confirmed below rather than assumed. `cal_size=0.25` carves calibration out of the remaining ~80%.

```
[3]: train_df, cal_df, test_df = occupation_train_cal_test_split(
    cleaned_training_rows, test_size=0.2, cal_size=0.25, random_state=42
)

print(f"Train occupations: {train_df['onetsoc_code'].nunique()}␣
    ↪({len(train_df)} rows)")
print(f"Cal occupations: {cal_df['onetsoc_code'].nunique()} ({len(cal_df)}␣
    ↪rows)")
print(f"Test occupations: {test_df['onetsoc_code'].nunique()} ({len(test_df)}␣
    ↪rows)")

train_codes = set(train_df["onetsoc_code"])
cal_codes = set(cal_df["onetsoc_code"])
test_codes = set(test_df["onetsoc_code"])

print(
    "\nPairwise disjoint (train/cal, train/test, cal/test):",
    train_codes.isdisjoint(cal_codes),
    train_codes.isdisjoint(test_codes),
    cal_codes.isdisjoint(test_codes),
)

# Confirm test_df matches notebook 03's two-way split exactly -- this is what
# keeps the Phase 1 baseline numbers valid after adding the conformal layer.
_, test_df_2way = occupation_train_test_split(
    cleaned_training_rows, test_size=0.2, random_state=42
)
print(
    "Test set identical to notebook 03's two-way split:",
    set(test_df["onetsoc_code"]) == set(test_df_2way["onetsoc_code"]),
)
```

```
Train occupations: 516 (2580 rows)
Cal occupations:   173 (865 rows)
Test occupations:  173 (865 rows)
```

```
Pairwise disjoint (train/cal, train/test, cal/test): True True True
Test set identical to notebook 03's two-way split: True
```

1.3 3. Train the baseline model on the (smaller) conformal train set

Note this model is trained on the ~75%-of-80% train split, not the full 80% used in notebook 03 — so a slightly different model than Phase 1's. Re-running `evaluate()` against `test_df` here re-confirms it still clearly beats naive, even with less training data, before trusting its calibration.

```
[4]: model = train_baseline_model(train_df, feature_cols)

phase1_style_result = evaluate(model, train_df, test_df, feature_cols)
print(phase1_style_result.summary())
```

```
Model RMSE (log-wage):  0.2626
Naive RMSE (log-wage):  0.4559
Model MAE (dollars):    $18,954
Naive MAE (dollars):    $31,959
Model improvement over naive: 40.7%
Test set: 865 rows across 173 held-out occupations
```

1.4 4. Calibrate: global split-conformal q_{hat}

Target 90% coverage ($\alpha=0.10$), calibrated on `cal_df` — data the model has never seen at training time.

```
[5]: alpha = 0.10
q_hat = calibrate(model, cal_df, feature_cols, alpha=alpha)
print(f"Global q_hat (log-wage space): {q_hat:.4f}")
print(f"Equivalent multiplicative factor in dollar space: exp(q_hat) = {np.
    ↳exp(q_hat):.3f}x")
```

```
Global q_hat (log-wage space): 0.4248
Equivalent multiplicative factor in dollar space: exp(q_hat) = 1.529x
```

1.5 5. Marginal coverage on held-out test occupations

Does the interval actually contain the true wage ~90% of the time? Reporting coverage AND mean interval width together — width is what tells us the interval is actually useful, not just wide enough to trivially cover.

Expected range: split conformal guarantees marginal coverage in $[1-\alpha, 1-\alpha + 1/(n_{\text{cal}}+1)]$ — close to 90% but not exactly, and that's correct, not an error to chase.

```
[6]: marginal_result = evaluate_coverage(model, test_df, feature_cols, q_hat,
    ↳target_coverage=1 - alpha)
print(marginal_result.summary())
```

```
'ALL': coverage=90.3% (target 90%), mean width=$69,017, n_occ=173
```

1.6 6. Conditional coverage by Job Zone

The most direct equity check: are lower Job Zone occupations (entry-level, less formal preparation — exactly the population this tool is meant to serve) systematically over- or under-covered relative to the 90% target?

Groups with fewer than 15 held-out occupations are flagged as low-confidence rather than hidden — a coverage number from a handful of occupations is noisy enough to be close to meaningless.

```
[7]: coverage_by_job_zone = evaluate_conditional_coverage(
      model, test_df, feature_cols, q_hat, group_col="job_zone",
      target_coverage=1 - alpha, min_n_occupations=15,
    )
coverage_by_job_zone
```

```
[7]:
```

	group_value	n_occupations	n_rows	coverage	mean_width_dollars \
0	2.0	64	320	0.946875	44210.800781
1	3.0	34	170	0.905882	61612.921875
2	4.0	40	200	0.850000	85072.546875
3	5.0	35	175	0.880000	103221.804688

	target_coverage	is_low_confidence
0	0.9	False
1	0.9	False
2	0.9	False
3	0.9	False

1.7 7. Conditional coverage by SOC major group

Checks whether whole occupational categories (e.g. healthcare vs. management vs. computer occupations) are miscalibrated.

```
[8]: test_df = test_df.copy()
test_df["soc_major_group"] = test_df["onetsoc_code"].map(soc_major_group)

coverage_by_soc_group = evaluate_conditional_coverage(
    model, test_df, feature_cols, q_hat, group_col="soc_major_group",
    target_coverage=1 - alpha, min_n_occupations=15,
)
coverage_by_soc_group
```

```
[8]:
```

	group_value	n_occupations	n_rows	coverage	mean_width_dollars \
0	11	10	50	0.540000	98075.570312
1	13	13	65	0.969231	76808.296875
2	15	6	30	1.000000	97960.500000
3	17	5	25	0.760000	91758.710938
4	19	11	55	0.963636	94754.281250
5	21	3	15	0.866667	69465.242188
6	23	1	5	1.000000	77836.484375
7	25	16	80	1.000000	72880.023438
8	27	9	45	0.844444	77301.781250
9	29	11	55	0.690909	115497.062500
10	31	3	15	1.000000	52186.628906
11	33	4	20	0.950000	65890.093750
12	35	3	15	1.000000	38171.500000
13	37	1	5	1.000000	39925.195312
14	39	7	35	0.857143	44559.582031

15	41	4	20	1.000000	65056.257812
16	43	9	45	0.977778	45540.933594
17	45	1	5	1.000000	47183.101562
18	47	14	70	0.900000	48904.910156
19	49	14	70	0.942857	51452.710938
20	51	15	75	0.960000	48793.312500
21	53	13	65	0.938462	54824.742188

	target_coverage	is_low_confidence
0	0.9	True
1	0.9	True
2	0.9	True
3	0.9	True
4	0.9	True
5	0.9	True
6	0.9	True
7	0.9	False
8	0.9	True
9	0.9	True
10	0.9	True
11	0.9	True
12	0.9	True
13	0.9	True
14	0.9	True
15	0.9	True
16	0.9	True
17	0.9	True
18	0.9	True
19	0.9	True
20	0.9	False
21	0.9	True

1.8 8. Conditional coverage by percentile level

Worth checking since `percentile` is a feature the melt introduced — residual magnitude may not be constant across the wage distribution even in log space (e.g. the 90th-percentile row might be harder to predict than the median row).

```
[9]: coverage_by_percentile = evaluate_conditional_coverage(
      model, test_df, feature_cols, q_hat, group_col="percentile",
      target_coverage=1 - alpha, min_n_occupations=15,
    )
coverage_by_percentile
```

```
[9]:   group_value  n_occupations  n_rows  coverage  mean_width_dollars  \
0         10           173      173  0.924855      40723.394531
1         25           173      173  0.930636      50372.937500
```

2	50	173	173	0.913295	65606.437500
3	75	173	173	0.878613	83593.304688
4	90	173	173	0.867052	104790.476562

	target_coverage	is_low_confidence
0	0.9	False
1	0.9	False
2	0.9	False
3	0.9	False
4	0.9	False

1.9 9. Decision point: is Mondrian calibration warranted?

Look at sections 6-8 above. If any subgroup (with adequate sample size, i.e. NOT flagged low-confidence) shows coverage meaningfully below the 90% target — say, below 85% — that’s a real, documented finding, not a bug to paper over. The cell below checks this programmatically against the Job Zone breakdown specifically, since that’s the subgroup most tied to the early-career population this project is meant to serve.

```
[10]: LOW_COVERAGE_THRESHOLD = 0.85

flagged_job_zones = coverage_by_job_zone[
    (coverage_by_job_zone["coverage"] < LOW_COVERAGE_THRESHOLD)
    & (~coverage_by_job_zone["is_low_confidence"])
]

if len(flagged_job_zones) > 0:
    print("Job Zone(s) with real undercoverage found -- applying Mondrian_
    ↪calibration:")
    print(flagged_job_zones[["group_value", "coverage", "n_occupations"]])
    apply_mondrian = True
else:
    print("No Job Zone shows real (adequately-sampled) undercoverage below "
          f"{LOW_COVERAGE_THRESHOLD:.0%}. Global calibration appears adequate; "
          "Mondrian is not applied by default (see conformal.py docstring on "
          "why it's a response to a finding, not a default).")
    apply_mondrian = False
```

No Job Zone shows real (adequately-sampled) undercoverage below 85%. Global calibration appears adequate; Mondrian is not applied by default (see conformal.py docstring on why it's a response to a finding, not a default).

1.10 10. Mondrian calibration (only runs if Section 9 found a real gap)

If applied: calibrates a separate $\hat{q}$ per Job Zone using only that Job Zone’s own calibration residuals, then re-evaluates conditional coverage by Job Zone with the group-specific $\hat{q}$ ’s. Reports before/after side by side — this comparison is the concrete evidence for the “honest uncertainty, and we correct it when it’s uneven” thesis, not just the rhetorical framing.

```
[11]: if apply_mondrian:
    q_hats_by_job_zone = calibrate_by_group(
        model, cal_df, feature_cols, group_col="job_zone", alpha=alpha
    )
    print("Per-Job-Zone q_hat (log-wage space):")
    for zone, q in sorted(q_hats_by_job_zone.items()):
        print(f"  Job Zone {zone}: q_hat={q:.4f} (global was {q_hat:.4f})")

    coverage_by_job_zone_mondrian = evaluate_conditional_coverage(
        model, test_df, feature_cols, q_hats_by_job_zone, group_col="job_zone",
        target_coverage=1 - alpha, min_n_occupations=15,
    )

    print("\nBEFORE (global q_hat):")
    print(coverage_by_job_zone[["group_value", "coverage",
↪ "mean_width_dollars", "n_occupations"]])
    print("\nAFTER (Mondrian, per-Job-Zone q_hat):")
    print(coverage_by_job_zone_mondrian[["group_value", "coverage",
↪ "mean_width_dollars", "n_occupations"]])
else:
    print("Skipped -- Section 9 found no adequately-sampled Job Zone below the "
          f"{LOW_COVERAGE_THRESHOLD:.0%} threshold.")
```

Skipped -- Section 9 found no adequately-sampled Job Zone below the 85% threshold.

1.11 11. Spot-check: interval for a previously wage-unmatched occupation

Same spirit as notebook 03's spot-check – but now with an honest uncertainty band attached, not just a point estimate. This is the actual deliverable the project's thesis is about: a range with a stated, validated confidence level, for exactly the occupations where no direct BLS lookup exists.

```
[12]: no_wage_match = feature_table[
    feature_table["a_median"].isna() & feature_table[list(skill_matrix.
↪ columns)].notna().all(axis=1)
]

if len(no_wage_match) > 0:
    sample_code_, sample_row = next(no_wage_match.head(3).iterrows())
    feature_values = {col: sample_row[col] for col in feature_cols if col !=
↪ "percentile"}
    x = pd.DataFrame([feature_values] * 5)
    x["percentile"] = [10, 25, 50, 75, 90]

    lo, hi = predict_interval(model, x, feature_cols, q_hat)
    point = np.exp(model.predict(x[feature_cols]))
```



```

print(f"{sample_row['title']} ({sample_code_}) -- with_
↪{int((1-alpha)*100)}% conformal interval:")
for p, l, pt, h in zip(x["percentile"], lo, point, hi):
    print(f"  {p}th percentile: point=${pt:,.0f}, interval=[${l:,.0f}, ${h:
↪,.0f}]")

```

Buyers and Purchasing Agents, Farm Products (13-1021.00) -- with 90% conformal interval:

```

10th percentile: point=$43,541, interval=[$28,472, $66,585]
25th percentile: point=$55,531, interval=[$36,313, $84,920]
50th percentile: point=$72,106, interval=[$47,152, $110,267]
75th percentile: point=$93,083, interval=[$60,869, $142,346]
90th percentile: point=$131,302, interval=[$85,861, $200,792]

```

1.12 12. Verdict

Fill in once the cells above have run against real data:

- **Marginal coverage:** [real number from Section 5] vs. 90% target. Mean interval width: [real number].
- **Conditional coverage:** [summarize what Sections 6-8 actually showed – which subgroups, if any, were under- or over-covered, and whether the gap was on an adequately-sampled group or one flagged low-confidence].
- **Mondrian applied?** [Yes/No, per Section 9's decision] – if yes, report the before/after coverage numbers from Section 10 explicitly.
- **Honest limitation to carry into the README:** [state whatever real gap remains after any correction – exactly as the 894-occupation gap and 5.9% crosswalk gap were stated plainly in Phase 0, rather than glossed over].

Phase 2 verdict: [proceed to Phase 3 (Explainability/SHAP) / needs another pass on X first].